

Rapport SAE 2.04

EXPLOITATION BASE DE DONNEE

MCD

MLD

SCRIPT SQL

27 MARS

Créé par :
FAYARD Gautier
CINAR Kamil
ALLIX Hugo



UNIVERSITÉ
SAVOIE
MONT BLANC

SOMMAIRE

INTRODUCTION.....	2
1. MCD / MLD ET SCRIPT	3
2. DESCRIPTION DES TABLES.....	3
2.1. BESOIN 1.....	3
2.1.1. TABLE PERSONNE.....	3
2.1.2. TABLE CONTRAT	3
2.1.3. TABLE TYPE CONTRAT	3
2.1.4. TABLE PROCEDURE.....	3
2.1.5. TABLE STATUT	4
2.1.6. TABLE ETAPE.....	4
2.2. BESOIN 2.....	4
2.2.1. TABLE PROJET	4
2.2.2. TABLE PUBLICATION	4
2.2.3. TABLE TYPE PUBLICATION.....	4
2.3. BESOIN 3.....	5
2.3.1. TABLE SALLE	5
2.3.2. TABLE EQUIPEMENT	5
3. EXPLICATION DES CONTRAINTES.....	5
3.1. BESOIN 1.....	5
3.1.1. CONTRAINTES SUR L'ID_HAL.....	5
3.1.2. CONTRAINTES SUR LES DIFFERENTS NOMS DE STATUTS	6
3.1.3. CONTRAINTES SUR LES DIFFERENTS STATUTS DES ETAPES	6
3.1.4. CONTRAINTES SUR LE TYPE DE CONTRAT ET LA DATE DE FIN	7
3.1.5. CONTRAINTES SUR LE TYPE DE CONTRAT ET LA DUREE DETERMINE	7
3.2. BESOIN 2.....	8
3.2.1. CONTRAINTES SUR L'ACCEPTATION D'UN PROJET	8
3.2.2. CONTRAINTES SUR LES ROLES DES PERSONNES	9
3.2.3. CONTRAINTES SUR LE TYPE DE CONTRAT ET LA DATE DE FIN	9
3.3. BESOIN 3.....	10
3.3.1. CONTRAINTES SUR LA CAPACITE DES SALLES	10
3.3.2. CONTRAINTES SUR LES DATES D'AFFECTATION	10
3.3.3. CONTRAINTES SUR LES TYPES DE SALLE	11
3.3.4. CONTRAINTES SUR L'IDENTIFIANT DE LA SALLE	12
3.3.5. CONTRAINTES SUR LE CODE USMB.....	12
CONCLUSION.....	13

Introduction

Dans le cadre du projet SAE 2.04, nous avons été chargés de concevoir et d'exploiter une base de données permettant la gestion d'un laboratoire de recherche. Le Laboratoire d'Informatique, Systèmes, Traitement de l'Information et de la Connaissance (LISTIC) de l'Université Savoie Mont Blanc est une unité de recherche nécessitant une solution de gestion informatisée pour optimiser le suivi des procédures administratives, du matériel et des différentes expérimentations. L'objectif principal est de proposer une base de données robuste et efficace pour centraliser les informations et permettre une meilleure gestion des ressources du laboratoire.

Ce rapport détaille les différentes étapes de la modélisation et de la mise en place de cette base de données. Il comprend la conception du Modèle Conceptuel des Données (MCD), du Modèle Logique des Données (MLD), la création des scripts SQL, ainsi que l'exploitation de la base pour répondre aux besoins définis. Nous examinerons également les contraintes mises en place pour assurer l'intégrité et la cohérence des données.

Répartition du travail

ALLIX Hugo : 33%

CINAR Kamil : 33%

FAYAR Gauthier : 33%

1. MCD / MLD et script

Les documents contenant le MCD, le MLD et les scripts associés ne sont pas inclus ici. Vous pourrez les retrouver dans des fichiers externes spécifiquement dédiés. Consultez ces documents pour accéder à tous les détails nécessaires.

2. Description des tables

2.1. Besoin 1

2.1.1. *Table Personne*

La table personne est composé de plusieurs lignes. Sa clé primaire est l'id-hal. Une personne est caractérisée par son nom, son prénom, un code INSEE, son courriel personnelle et professionnel et enfin il est interne ou externe au laboratoire. La table personne est lié à la table Contrat avec un lien 1,n car une personne peut signer plusieurs contrats. Il est également lié à la table Salle avec un lien 1,n car une personne peut se voir affecter plusieurs salles. Il a aussi un lien 0,n avec la table Projet car peut avoir aucun rôle dans aucun projet ou à l'inverse plusieurs. Et enfin, il est lié à la table Publication avec un lien 0,n car peut publier 0 ou plusieurs publications.

2.1.2. *Table Contrat*

La table Contrat contient plusieurs enregistrements. Son identifiant unique est le Num_Contrat. Chaque contrat est défini par une date de début, une date de fin et une indication sur la durée déterminée ou non. Elle est reliée à la table Personne par une relation 1,1, puisqu'un contrat est signé par une seule personne. Elle est aussi connectée à la table Type_Contrat via une relation 0,n, car un type de contrat peut être associé à plusieurs contrats.

2.1.3. *Table Type contrat*

La table Type_Contrat référence différents types de contrats. Son identifiant unique est l'ID_TypeContrat. Chaque type de contrat est défini par un libellé. Elle est reliée à la table Contrat par une relation 0,n, permettant ainsi à un type de contrat d'être lié à plusieurs contrats ou à aucun.

2.1.4. *Table Procédure*

La table Procédure regroupe diverses procédures. Son identifiant principal est l'ID_Procédure. Chaque procédure se caractérise par un nom, une description et un ordre numérique. Elle est en relation 1,n avec la table Étape, car une procédure peut comporter plusieurs étapes. De plus, elle entretient une relation 0,n avec elle-même, permettant d'établir des dépendances entre procédures.

2.1.5. Table Statut

La table Statut regroupe plusieurs entrées. Sa clé primaire est l'Id_statut. Chaque statut possède un nom spécifique. Elle est en relation 1,1 avec la table Étape, car chaque étape est obligatoirement associée à un statut unique.

2.1.6. Table Etape

La table Étape liste différentes étapes du processus. Sa clé primaire est l'ID_Étape. Chaque étape est décrite par une date de mise à jour et un statut. Elle est connectée à la table Procédure par une relation 1,1, chaque étape étant intégrée dans une seule procédure. Enfin, elle est liée à la table Statut en 1,1, assurant qu'une étape possède toujours un statut bien défini.

2.2. Besoin 2

2.2.1. Table Projet

La table Projet recense plusieurs projets. Son identifiant unique est l'IdProjet. Chaque projet est défini par un nom, une date de dépôt au guichet, un nom de guichet, une date de début, une durée et un budget. Elle est reliée à la table Personne par une relation 0,n, car une personne peut ne participer à aucun projet ou être impliquée dans plusieurs. De plus, un projet peut être associé à diverses publications via une relation 0,n avec la table Publication.

2.2.2. Table Publication

La table Publication stocke différentes publications. Sa clé primaire est l'IdPublication. Chaque publication est identifiée par le nom du journal ou de la conférence dans laquelle elle a été publiée. Elle est liée à la table Personne par une relation 0,n, puisqu'une personne peut ne publier aucun article ou en rédiger plusieurs. Elle est également en relation 0,n avec la table Projet, car une publication peut être associée à plusieurs projets ou à aucun.

2.2.3. Table Type publication

La table Type publication répertorie les différentes catégories de publications. Son identifiant principal est l'Id_TypePubli. Chaque type de publication est décrit par son intitulé, son rang et son moyen d'édition. Elle est connectée à la table Publication par une relation 0,n, signifiant qu'un type de publication peut être associé à plusieurs publications.

2.3. Besoin 3

2.3.1. Table Salle

La table Salle contient plusieurs enregistrements. Son identifiant unique est ID_SALLE. Chaque salle est définie par un numéro d'étage, un type de salle et sa capacité d'accueil. Elle est reliée à la table Equipement par une relation 1,n, car une salle peut être associée à plusieurs équipements. Elle est aussi connectée à la table Affectation via une relation 1,n, car une salle peut être affectée à plusieurs personnes.

2.3.2. Table Equipement

La table Equipement contient plusieurs enregistrements. Son identifiant unique est CODE_USMB. Chaque équipement est défini par un code unique, un type, et une association à une salle spécifique via ID_SALLE. Elle est reliée à la table Utilise par une relation 1,n, car un équipement peut être utilisé dans plusieurs projets. Elle est aussi connectée à la table Salle via une relation n,1, car un équipement est affecté à une seule salle.

3. Explication des contraintes

3.1. Besoin 1

3.1.1. Contrainte sur l'id_hal

Cette contrainte vise à assurer la conformité de l'id_hal. En effet, l'id_hal doit commencer par « hal- » puis être suivi de 7 ou 8 chiffres et chaque code doivent être unique c'est la raison pour laquelle id_hal est la clé primaire.

```
ALTER TABLE PERSONNE
ADD CONSTRAINT CK_PERSONNE_ID_HAL
CHECK (ID_HAL ~ '^hal-\d{7,8}$');
```

La contrainte exige qu'id_hal commence par « hal- » est suivi d'une suite de 7 ou 8 chiffres.

Insertion fonctionnelle :

```
INSERT INTO PERSONNE(ID_HAL) VALUES ('hal-1234567')
```

Insertion non fonctionnelle :

```
INSERT INTO PERSONNE(ID_HAL) VALUES ('hal-12345')
```

Ici, l'insertion est impossible car même si ID_HAL commence bien par hal- il faut qu'il soit suivi de 7 ou 8 chiffres or ce n'est pas le cas ici.

3.1.2. Contrainte sur les différents noms de statuts

Cette contrainte a pour but s'assurer que les noms de statut sont bien conformes au statut existant. Ici, nom de statut disponibles sont « stagiaire », « étudiant doctorant », « post-doctorant », « BIATSS » ou « enseignant-chercheur », il faut donc que la saisie de n'importe quel autre nom de statut soit impossible.

```
ALTER TABLE STATUT
ADD CONSTRAINT CK_STATUT_NOM_STATUT
CHECK (NOM_STATUT IN ('stagiaire', 'étudiant doctorant', 'post-doctorant', 'BIATSS', 'enseignant-chercheur'));
```

L'utilisation de la fonction **IN** signifie que nom_statut doit être compris dans les différents termes entre parenthèses.

Insertion fonctionnelle :

```
INSERT INTO STATUT (ID_STATUT, ID_TYPECONTRAT, NOM_STATUT) VALUES (1, 2, 'stagiaire');
```

Insertion non fonctionnelle :

```
INSERT INTO STATUT (ID_STATUT, ID_TYPECONTRAT, NOM_STATUT) VALUES (2, 3, 'manager');
```

Manager n'est pas reconnu comme nom de statut, l'insertion est impossible.

3.1.3. Contrainte sur les différents statuts des étapes

Cette contrainte doit comme pour la précédente doit s'assurer que les statuts de étapes soient bien conformes. Les statuts d'étape sont « démarré », « en signature » et « terminé ».

```
ALTER TABLE ETAPE
ADD CONSTRAINT CK_STATUE_ETAPE
CHECK (STATUE_ETAPE IN ('démarré', 'en signature', 'terminé'));
```

Ici, la contrainte fonctionne comme pour la précédente avec la fonction **IN**.

Insertion fonctionnelle :

```
INSERT INTO ETAPE (ID_ETAPE, ID_PROCEDURE, NUM_CONTRAT, STATUE_ETAPE)
VALUES (1, 2, 3, 'démarré');
```

Insertion non fonctionnelle :

```
INSERT INTO ETAPE (ID_ETAPE, ID_PROCEDURE, NUM_CONTRAT, STATUE_ETAPE)
VALUES (2, 2, 3, 'en cours');
```

« En cours » ne fait pas partie des statuts des étapes possibles.

3.1.4. *Contrainte sur le type de contrat et la date de fin*

Cette contrainte doit faire en sorte que si le l'id_typeContrat est « 1 » ce qui correspond à un contrat à durée déterminé alors le date de fin de la table personne ne doit pas être nulle et supérieur à la date de début. En revanche s'il n'est pas 1 alors la date de fin doit être nulle.

```
ALTER TABLE CONTRAT
ADD CONSTRAINT CK_CONTRAT_DATEFIN_RULE
CHECK (((ID_TYPECONTRAT = 1 AND DATEFIN IS NOT NULL AND DATEFIN > DATEDEBUT) OR
(ID_TYPECONTRAT != 1 AND DATEFIN IS NULL)));
```

On retrouve ici bien id_typecontrat est égal à 1 et date de fin pas nulle et supérieur à la date de début ou id_typecontrat n'est pas égal à 1 et dans ce cas la date de fin est nulle.

Insertion fonctionnelle :

```
INSERT INTO CONTRAT (NUM_CONTRAT, ID_HAL, ID_STATUT, ID_TYPECONTRAT, DATEDEBUT,
DATEFIN) VALUES (1, 'hal-12345', 2, 1, '2025-01-01', '2025-12-31');
```

Insertion non fonctionnelle :

```
INSERT INTO CONTRAT (NUM_CONTRAT, ID_HAL, ID_STATUT, ID_TYPECONTRAT, DATEDEBUT,
DATEFIN) VALUES (2, 'hal-12345', 2, 1, '2025-01-01', NULL);
```

L'ID statut est 1 or 1 est pour les contrats à durée déterminé donc date de fin ne peut pas être nulle.

3.1.5. *Contrainte sur le type de contrat et la durée déterminé*

Cette contrainte s'assure de la cohérence entre le type de contrat et si la durée est déterminée. Si la durée est déterminée, alors il faut qu'id_typecontrat soit 1 sinon, il faut que l'id_typecontrat soit 2 ou 3.

```
ALTER TABLE CONTRAT
ADD CONSTRAINT CK_CONTRAT_DUREE_TYPE
CHECK (((DUREE_EST_DETERMINE = TRUE AND ID_TYPECONTRAT = 1) OR
((DUREE_EST_DETERMINE = FALSE OR DUREE_EST_DETERMINE IS NULL) AND (ID_TYPECONTRAT
= 2 OR ID_TYPECONTRAT = 3))));
```


Nous avons aussi ajouté la possibilité que `duree_est_determine` soit nulle si `id_type_contrat` soit 2 ou 3.

Insertion fonctionnelle :

```
INSERT INTO CONTRAT (NUM_CONTRAT, ID_HAL, ID_STATUT, ID_TYPECONTRAT, DATEDEBUT,
DATEFIN) VALUES (2, 'hal-123456', 3, 1, '2025-01-01', '2025-12-31');
```

Insertion non fonctionnelle :

```
INSERT INTO CONTRAT (NUM_CONTRAT, ID_HAL, ID_STATUT, ID_TYPECONTRAT,
DUREE_EST_DETERMINE) VALUES (4, 'hal-12345', 2, 2, TRUE);
```

L'ID du statut est 2 est `DUREE_EST_DETERMINE` est vrai ce qui est impossible car seul les ID à 1 correspondent au contrat à durée déterminé.

3.2. Besoin 2

3.2.1. Contrainte sur l'acceptation d'un projet

Cette contrainte vise à garantir la cohérence des informations liées à un projet. En effet, si un projet est renseigné, alors la date de début, la durée prévisionnelle et le budget doivent tous être non nuls. À l'inverse, si le projet n'a pas de date de début, alors les autres champs doivent également être nuls.

```
ALTER TABLE PROJET
ADD CONSTRAINT CK_PROJET_ACCEPTE
CHECK ((DATEDEBUTPROJET IS NOT NULL AND DUREE_PREVISIONNELLE IS NOT NULL AND
BUDGET IS NOT NULL)
OR (DATEDEBUTPROJET IS NULL AND DUREE_PREVISIONNELLE IS NULL AND BUDGET IS NULL));
```

Insertion fonctionnelle :

```
INSERT INTO PROJET (DATEDEBUTPROJET, DUREE_PREVISIONNELLE, BUDGET)
VALUES ('2025-06-01', 12, 50000);
```

Insertion non fonctionnelle :

```
INSERT INTO PROJET (DATEDEBUTPROJET, DUREE_PREVISIONNELLE, BUDGET)
VALUES ('2025-06-01', NULL, 50000);
```

Ici, `DATEDEBUTPROJET` est renseignée, mais `DUREE_PREVISIONNELLE` est NULL, ce qui viole la contrainte.

3.2.2. Contrainte sur les rôles des personnes

Cette contrainte garantit que le rôle attribué à une personne est bien parmi ceux prédéfinis dans le système.

C'est-à-dire : Chef de projet, Rédacteur, Coordinateur, Développeur, Testeur, Consultant, Responsable de la communication, Administrateur.

```
Alter table ROLE_PERSONNE

ADD CONSTRAINT CK_ROLE_PERSONNE

CHECK (ROLE IN ('Chef de projet', 'Rédacteur', 'Coordinateur', 'Développeur', 'Testeur',
'Consultant', 'Responsable de la communication', 'Administrateur'));
```

Insertion fonctionnelle :

```
INSERT INTO ROLE_PERSONNE (IDPROJET, ID_HAL, ROLE) VALUES (1, 'hal-12348', 'Chef de projet');
```

Insertion non fonctionnelle :

```
INSERT INTO ROLE_PERSONNE (IDPROJET, ID_HAL, ROLE) VALUES (1, 'hal-12348', 'Manageur');
```

Cette requête ne fonctionne pas, car "**Manageur**" ne fait pas partie de la liste des valeurs autorisées par la contrainte.

3.2.3. Contrainte sur le type de contrat et la date de fin

Cette contrainte permet de s'assurer que le rang d'une publication se situe bien de 0 à 5, ni plus ni moins.

```
Alter table type_PUBLICATION

ADD CONSTRAINT CK_PUBLICATION_RANG

CHECK (RANG_PUBLICATION BETWEEN 0 AND 5);
```

Insertion fonctionnelle :

```
INSERT INTO TYPE_PUBLICATION(ID_TYPEPUBLI, RANG_PUBLICATION) VALUES (2,4);
```

Insertion non fonctionnelle :

```
INSERT INTO TYPE_PUBLICATION(ID_TYPEPUBLI, RANG_PUBLICATION) VALUES (2,6);
```

Cette requête ne fonctionne pas, car la note est **supérieure à 5**.

3.3. Besoin 3

3.3.1. *Contrainte sur la capacité des salles*

Cette contrainte vérifie que la capacité des salles (la colonne `CAPACITE`) soit comprise entre 0 et 300. Cela empêche d'ajouter une salle avec une capacité illogique, comme une salle ayant une capacité négative ou supérieure à 300.

```
ALTER TABLE SALLE  
  
ADD CONSTRAINT CHECK_CAPACITE_SALLE  
  
CHECK (CAPACITE >= 0 AND CAPACITE <= 300);
```

Insertion fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE) VALUES ('A101', 1, 'bureau', 100);
```

Insertion non fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE) VALUES ('B103', 2, 'bureau', 350);
```

Ici, la capacité est **supérieure à 300** donc l'insertion ne fonctionnera pas

3.3.2. *Contrainte sur les dates d'affectation*

Cette contrainte impose des règles sur les dates d'affectation (`DATE_AFFECTATION`) et de fin d'affectation (`DATE_FIN`). Elle garantit que :

Si la date de fin (`DATE_FIN`) est `NULL`, la date d'affectation (`DATE_AFFECTATION`) doit être renseignée.

Si une date de fin est spécifiée, elle doit être postérieure à la date d'affectation (ce qui a du sens, car une personne ne peut pas être affectée après son départ).

```
ALTER TABLE AFFECTATION
ADD CONSTRAINT CK_AFFECTATION_DATEFIN_RULE
CHECK ((DATE_FIN IS NULL AND DATE_AFFECTATION IS NOT NULL) OR
      (DATE_FIN IS NOT NULL AND DATE_FIN > DATE_AFFECTATION AND DATE_AFFECTATION IS
      NOT NULL));
```

Insertion fonctionnelle :

```
INSERT INTO AFFECTATION (ID_SALLE, ID_HAL, DATE_AFFECTATION, DATE_FIN)
VALUES ('B102', 'P001', '2024-01-10', '2024-02-10');
```

Insertion non fonctionnelle :

```
INSERT INTO AFFECTATION (ID_SALLE, ID_HAL, DATE_AFFECTATION, DATE_FIN)
VALUES ('B102', 'P002', '2024-03-01', '2024-02-28');
```

La date de fin étant **antérieur à la date d'affectation**, l'insertion ne sera pas possible.

3.3.3. *Contrainte sur les types de salle*

Cette contrainte assure que la valeur de la colonne `TYPE_SALLE` soit l'un des types spécifiés : 'technique', 'bureau', 'stockage', 'archivage', 'blanche', 'expérimentation', 'serveur', ou 'reunion'. Cela garantit que les types de salle sont limités et cohérents avec les besoins du laboratoire.

```
ALTER TABLE SALLE
ADD CONSTRAINT CHECK_TYPE_SALLE
CHECK (TYPE_SALLE IN ('technique', 'bureau', 'stockage', 'archivage', 'blanche',
'expérimentation', 'serveur', 'reunion'));
```

Insertion fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE) VALUES ('C201', 2, 'reunion', 50);
```

Insertion non fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE)
VALUES ('C202', 2, 'votenzlagence', 50);
```

Ici, 'votezlagence' n'est pas un type de salle reconnu donc ne peut pas être inséré.

3.3.4. Contrainte sur l'identifiant de la salle

Cette contrainte vérifie que l'identifiant de la salle (ID_SALLE) suit un format spécifique. Il doit commencer par une lettre majuscule ou minuscule suivie de trois chiffres. Par exemple, A001 ou B123. Cela garantit que les identifiants de salle sont toujours bien structurés.

```
ALTER TABLE SALLE
ADD CONSTRAINT CHECK_ID_SALLE_FORMAT
CHECK (ID_SALLE ~ '^[A-Za-z][0-9]{3}$');
```

Insertion fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE) VALUES ('D300', 3, 'stockage', 80);
```

Insertion non fonctionnelle :

```
INSERT INTO SALLE (ID_SALLE, NUM_ETAGE, TYPE_SALLE, CAPACITE) VALUES ('300D', 3, 'stockage', 80);
```

L'id de la salle ne correspond pas au format requis qui est une lettre puis 3 chiffres donc l'insertion est impossible.

3.3.5. Contrainte sur le code USMB

Cette contrainte garantit que le code USMB de chaque équipement (CODE_USMB) est composé exactement de 10 chiffres. Cela assure que chaque équipement a un code unique et valide, ce qui peut être crucial pour le suivi et la gestion des équipements dans le laboratoire.

```
ALTER TABLE EQUIPEMENT
ADD CONSTRAINT CK_CODE_USMB_FORMAT
CHECK (CODE_USMB ~ '[0-9]{10}$');
```

Insertion fonctionnelle :

```
INSERT INTO EQUIPEMENT (NOM_EQUIPEMENT, ID_SALLE, TYPE, CODE_USMB)  
VALUES ('Serveur', 'B102', 'informatique', '9876543210');
```

Insertion non fonctionnelle :

```
INSERT INTO EQUIPEMENT (NOM_EQUIPEMENT, ID_SALLE, TYPE, CODE_USMB)  
VALUES ('Serveur', 'B102', 'informatique', '9876543');
```

Ici, le code usmb est constitué de chiffre pas 10 donc l'insertion est impossible car il faut une suite d'exactly 10 chiffres.

Conclusion

La mise en place de cette base de données a permis d'optimiser la gestion du laboratoire LISTIC en structurant efficacement les informations relatives aux personnes, aux contrats, aux projets et aux équipements. Grâce à une conception rigoureuse et à l'application de contraintes d'intégrité adaptées, nous avons pu garantir la fiabilité des données et assurer un suivi précis des différentes activités du laboratoire.

Les résultats obtenus à travers l'exploitation de la base démontrent l'efficacité de notre modèle et sa capacité à répondre aux besoins du laboratoire. Cette solution pourra être utilisée comme socle pour d'éventuelles évolutions futures, notamment en intégrant des fonctionnalités avancées comme l'analyse des consommations énergétiques ou le suivi des publications scientifiques en temps réel. Cette base de données constitue ainsi un outil performant et évolutif pour la gestion du LISTIC.